

Resumo das duas últimas atualizações (passageiro)

Documento de referência das entregas no perfil **passageiro** (/p). Para PDF: abra este ficheiro no VS Code/Cursor (pré-visualização Markdown), no browser, ou exporte a partir de Word/Google Docs — **Imprimir** → **Guardar como PDF**.

1. Retirar a obrigatoriedade de login (apenas passageiro)

Objetivo

O passageiro **não passa pela tela de login** com e-mail/senha; a app obtém automaticamente um **JWT de convidado** (sessão anónima por navegador), porque o backend continua a exigir utilizador e token para viagem e WebSocket.

Comportamento

- **Raiz /** : sem sessão guardada, redireciona para **/p** em vez de **/login** (apps/web/src/app/page.tsx).
- **/p** : remove `requireAuth` + redirect para **/login** ; se existir sessão **ADMIN** ou **DRIVER**, redireciona para **/admin** ou **/d** ; caso contrário chama `ensurePassengerSession()` (apps/web/src/lib/auth/passengerSession.ts).
- **Sessão anónima**: `localStorage` guarda `dreamdriver_passenger_guest_id` (UUID); **POST** `/auth/passenger-session` com `{ guestId }` cria ou reutiliza utilizador `guest_<uuid>@passenger.local` com papel **PASSENGER** e devolve `{ token, user }` (JWT longo, ex.: 365 dias).
- **API**: `AuthService.passengerSession` (apps/api/src/auth/auth.service.ts) + rota pública **POST** `/auth/passenger-session` (apps/api/src/auth/auth.controller.ts).
- **Motorista e admin** continuam a usar **/login** e `requireAuth` nas respetivas páginas.

Ficheiros principais

Área	Ficheiros
API	apps/api/src/auth/auth.service.ts, apps/api/src/auth/auth.controller.ts
Web	apps/web/src/lib/auth/passengerSession.ts, apps/web/src/lib/auth/index.ts, apps/web/src/lib/auth.ts, apps/web/src/app/page.tsx, apps/web/src/app/p/page.tsx

2. “Retirar” o check-in para o passageiro (só acompanhar a van)

Objetivo

Desativar a interface de check-in (botões, lista “Passageiros por ponto”, marcadores de passageiros/pontos no mapa) **sem apagar** a lógica nem os endpoints da API.

Comportamento

- Variável `NEXT_PUBLIC_PASSENGER_CHECKIN_ENABLED` : só com valor **true** a UI de check-in é mostrada; em qualquer outro caso (omissão ou **false**), o passageiro vê apenas **status da viagem**, **Seguir van** e o **mapa com a van** (`isPassengerCheckinEnabled()` em apps/web/src/lib/env.ts).

- Com check-in desligado: não se pede `/checkins/pickup-points` ; o snapshot não alimenta estado de check-ins para o mapa; eventos socket de check-in não atualizam lista visível.
- **API** (`POST /checkins` , etc.) **inalterada**; reativação = definir `NEXT_PUBLIC_PASSENGER_CHECKIN_ENABLED=true` e reiniciar o Next.js.

Ficheiros principais

Área	Ficheiros
Web	<code>apps/web/src/lib/env.ts</code> , <code>apps/web/src/app/p/page.tsx</code>
Doc env	<code>.env.example</code> (comentário sobre a variável)

Fluxo resumido (sessão + modo só mapa)

1. Utilizador abre `/` ou `/p` sem ser admin/motorista.
2. É gerido `guestId` no `localStorage` e chamado `POST /auth/passenger-session` .
3. Com JWT de passageiro, a app obtém estado da viagem e posição da van (snapshot + WebSocket).
4. A UI de check-in só aparece se `NEXT_PUBLIC_PASSENGER_CHECKIN_ENABLED=true` .

Como gerar o PDF deste documento

1. **Cursor / VS Code**: pré-visualização Markdown → imprimir para PDF (se o sistema suportar).
2. **Word ou Google Docs**: copiar o conteúdo deste `.md` ou abrir ficheiro convertido → **Ficheiro** → **Transferir como PDF**.
3. **Browser**: arrastar o ficheiro para uma extensão de Markdown ou colar o HTML exportado → **Imprimir** → **Guardar como PDF**.